Operating Systems – 3 easy pieces

Chap. 36: I/O Devices

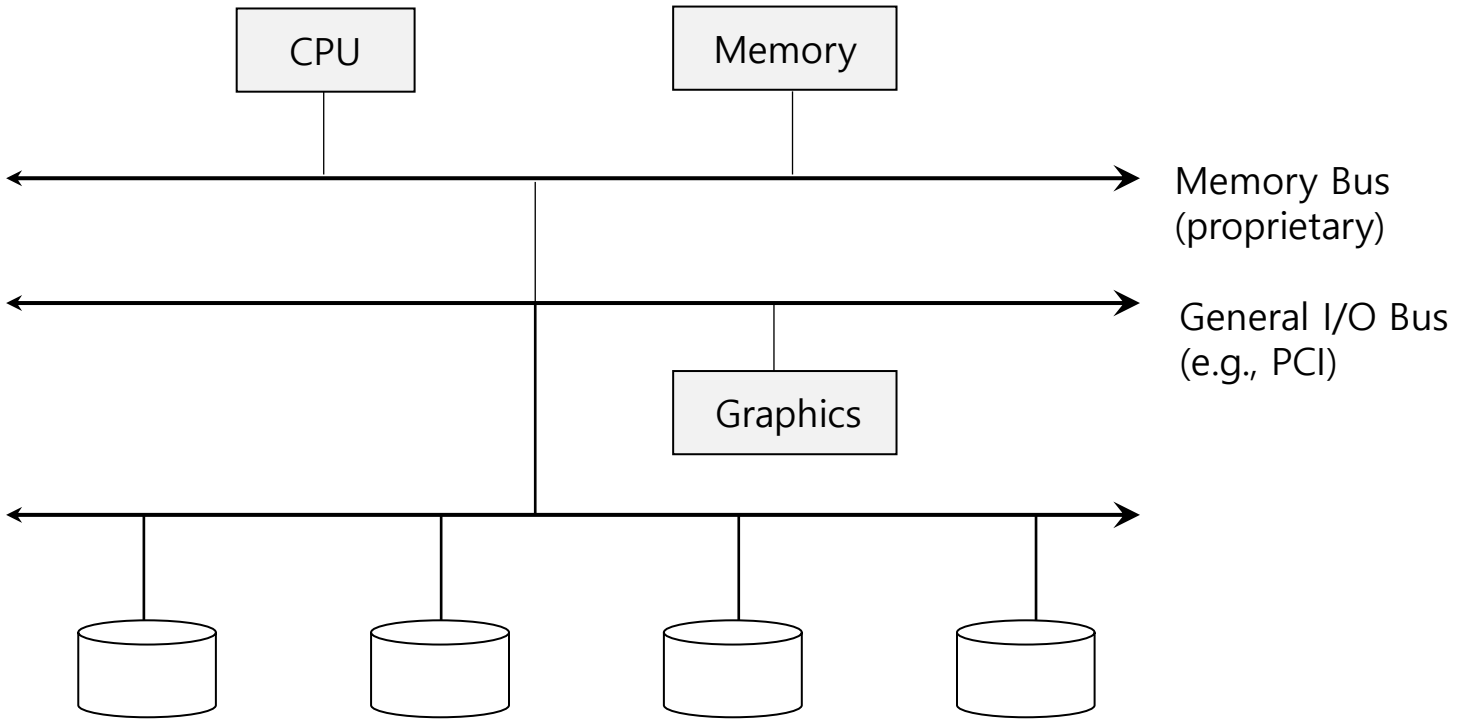
Dr. B. Wajid

Wednesday: Aug. 28, 2024

"When you do something with a lot of honesty, appetite and commitment, the input reflects in the output." – A.R. Rahman.

– Inspiration: Imagine a program without any input (it produces the same result each time); now imagine a program with no output (what was the purpose of it running?)

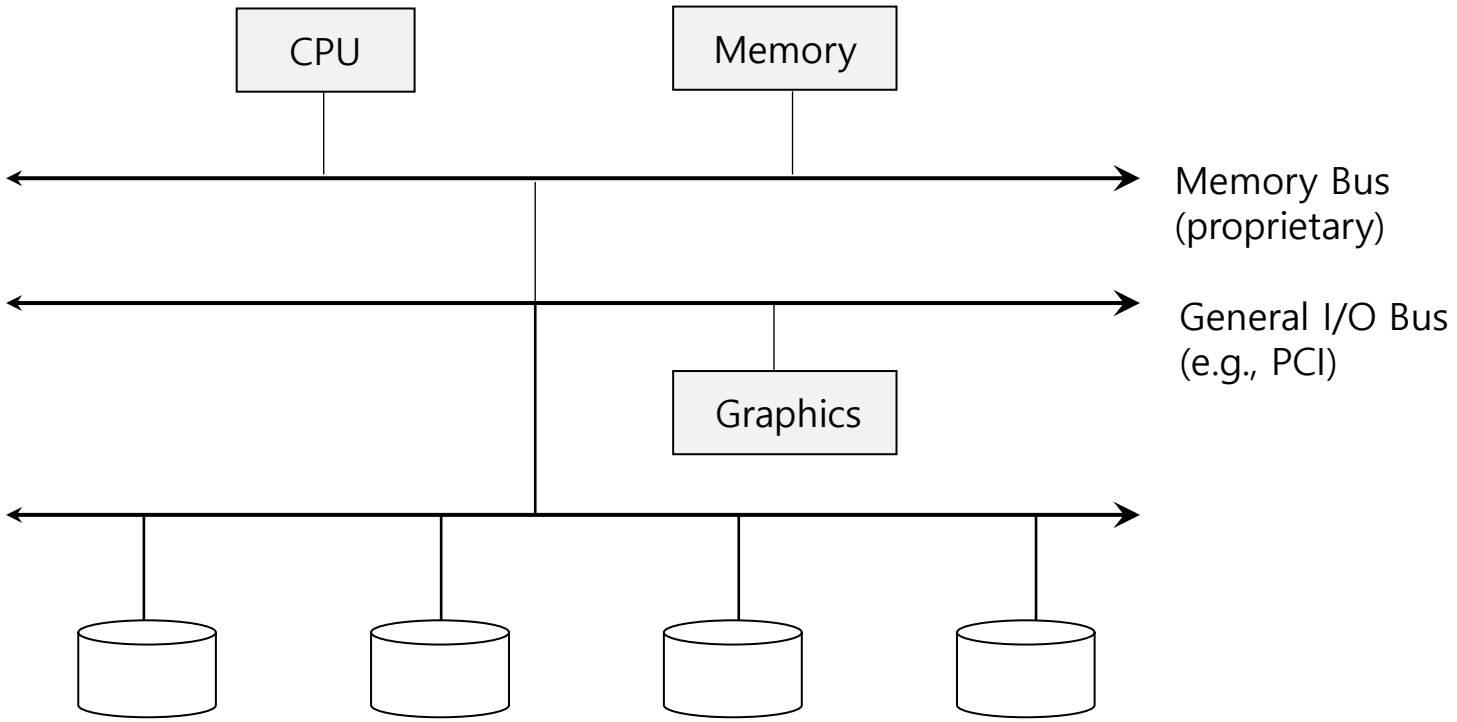
Prototypical System Architecture



Hierarchical Structure: Why do we need such a structure?

- Simply: physics, and cost.
- The faster the bus, the shorter it must be. Thus high-performance memory bus does not have much room to plug devices and is quite costly.
- Components that demand high performance (such as the graphics card) are nearer the CPU.
- Lower performance components are further away.
- Placing disks and other slow devices on a peripheral bus are many, for instance one can place many devices on it.

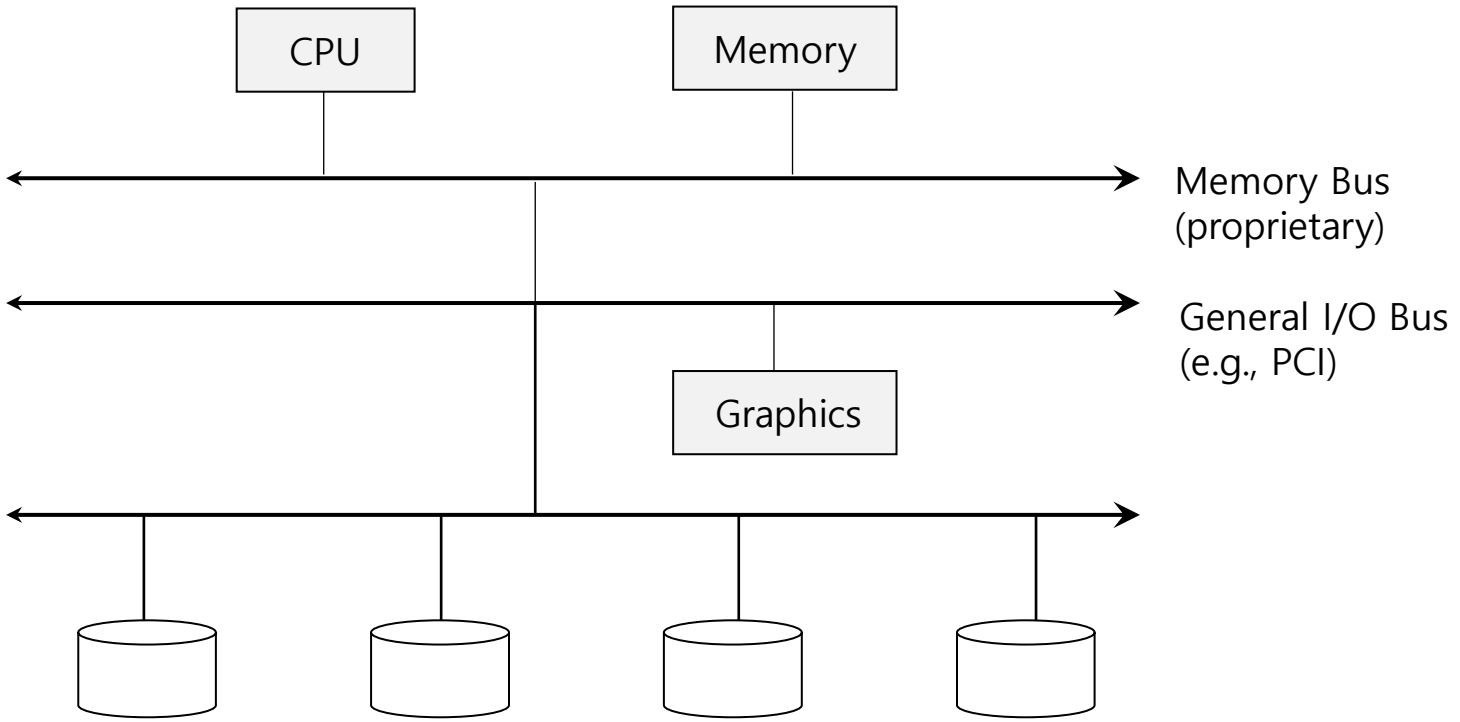
Prototypical System Architecture



Hierarchical Structure: Why do we need such a structure?

- Simply: physics, and cost.
- The faster the bus, the shorter it must be. Thus high-performance memory bus does not have much room to plug devices and is quite costly.
- Components that demand high performance (such as the graphics card) are nearer the CPU.
- Lower performance components are further away.
- Placing disks and other slow devices on a peripheral bus are many, for instance one can place many devices on it.

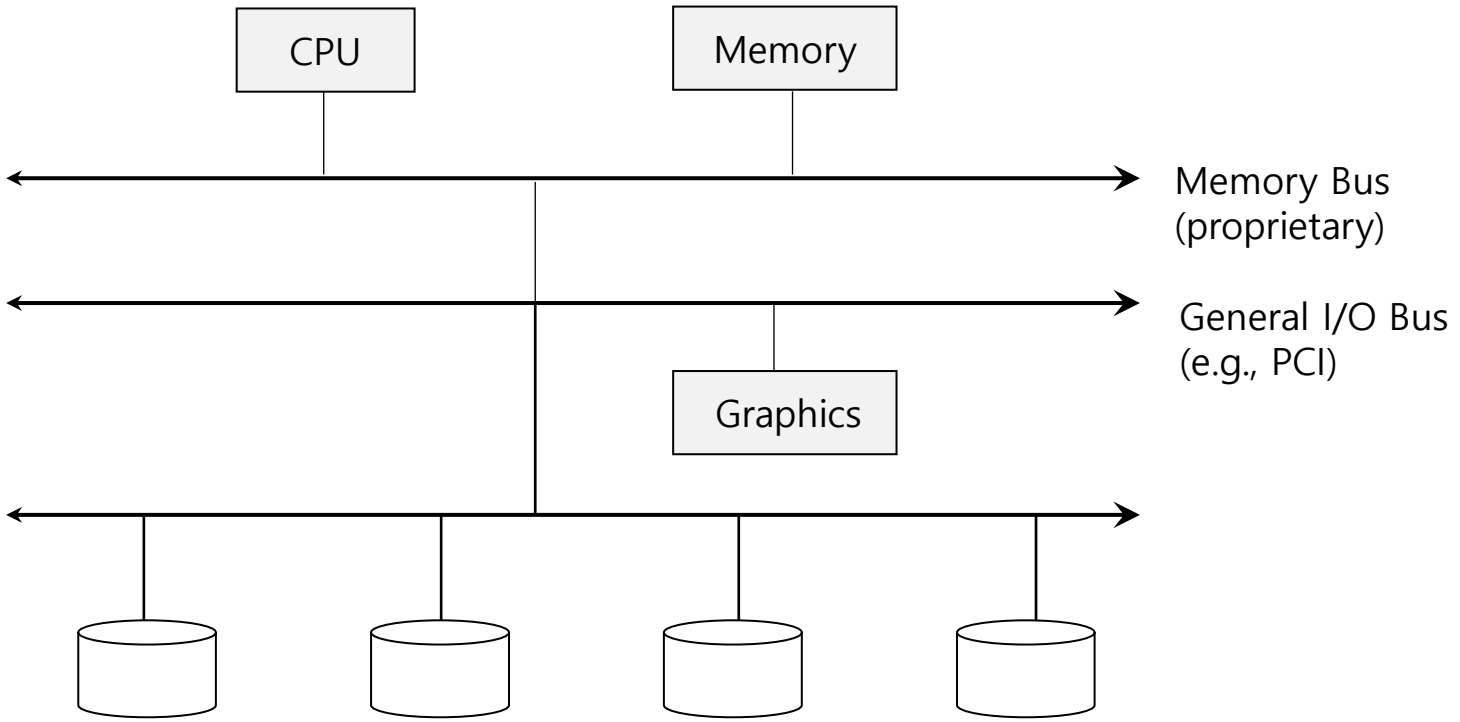
Prototypical System Architecture



Hierarchical Structure: Why do we need such a structure?

- Simply: physics, and cost.
- The faster the bus, the shorter it must be. Thus high-performance memory bus does not have much room to plug devices and is quite costly.
- Components that demand high performance (such as the graphics card) are nearer the CPU.
- Lower performance components are further away.
- Placing disks and other slow devices on a peripheral bus are many, for instance one can place many devices on it.

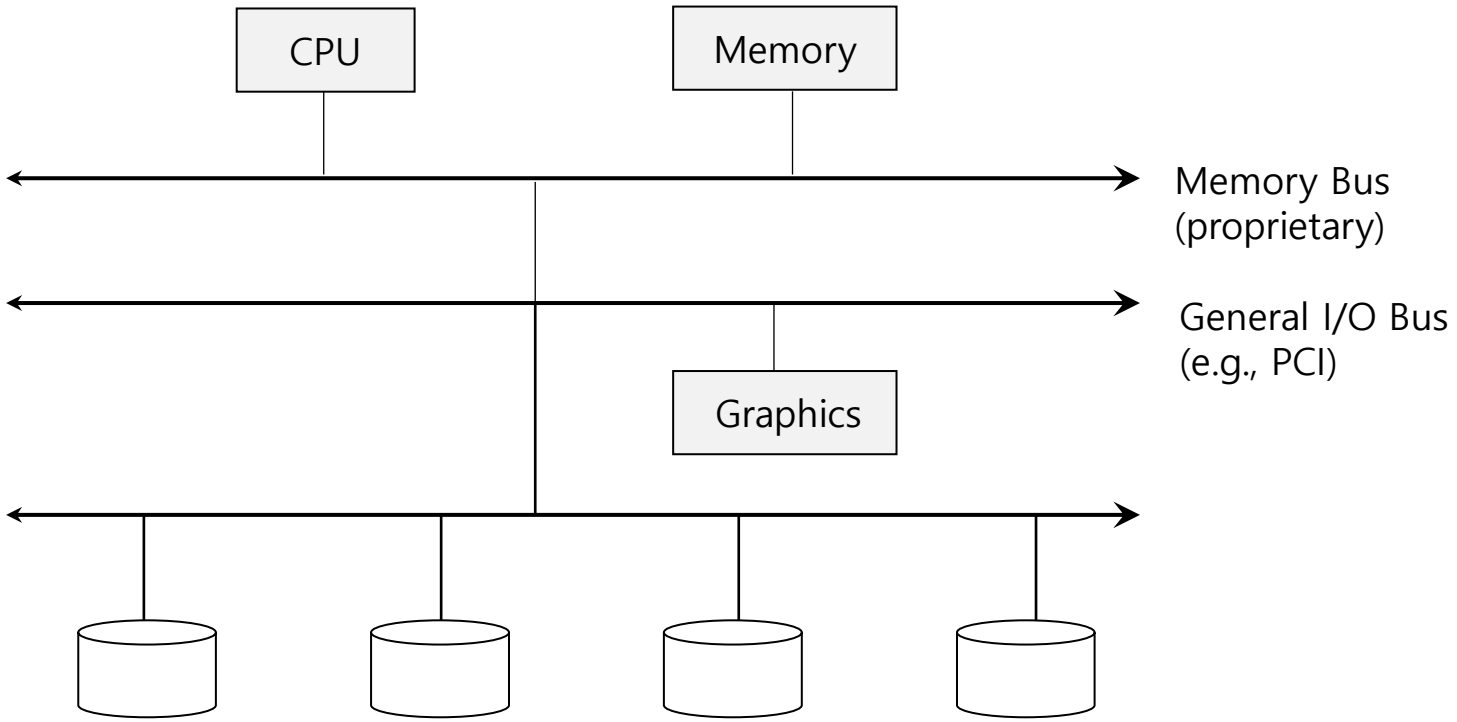
Prototypical System Architecture



Hierarchical Structure: Why do we need such a structure?

- Simply: physics, and cost.
- The faster the bus, the shorter it must be. Thus high-performance memory bus does not have much room to plug devices and is quite costly.
- Components that demand high performance (such as the graphics card) are nearer the CPU.
- Lower performance components are further away.
- Placing disks and other slow devices on a peripheral bus are many, for instance one can place many devices on it.

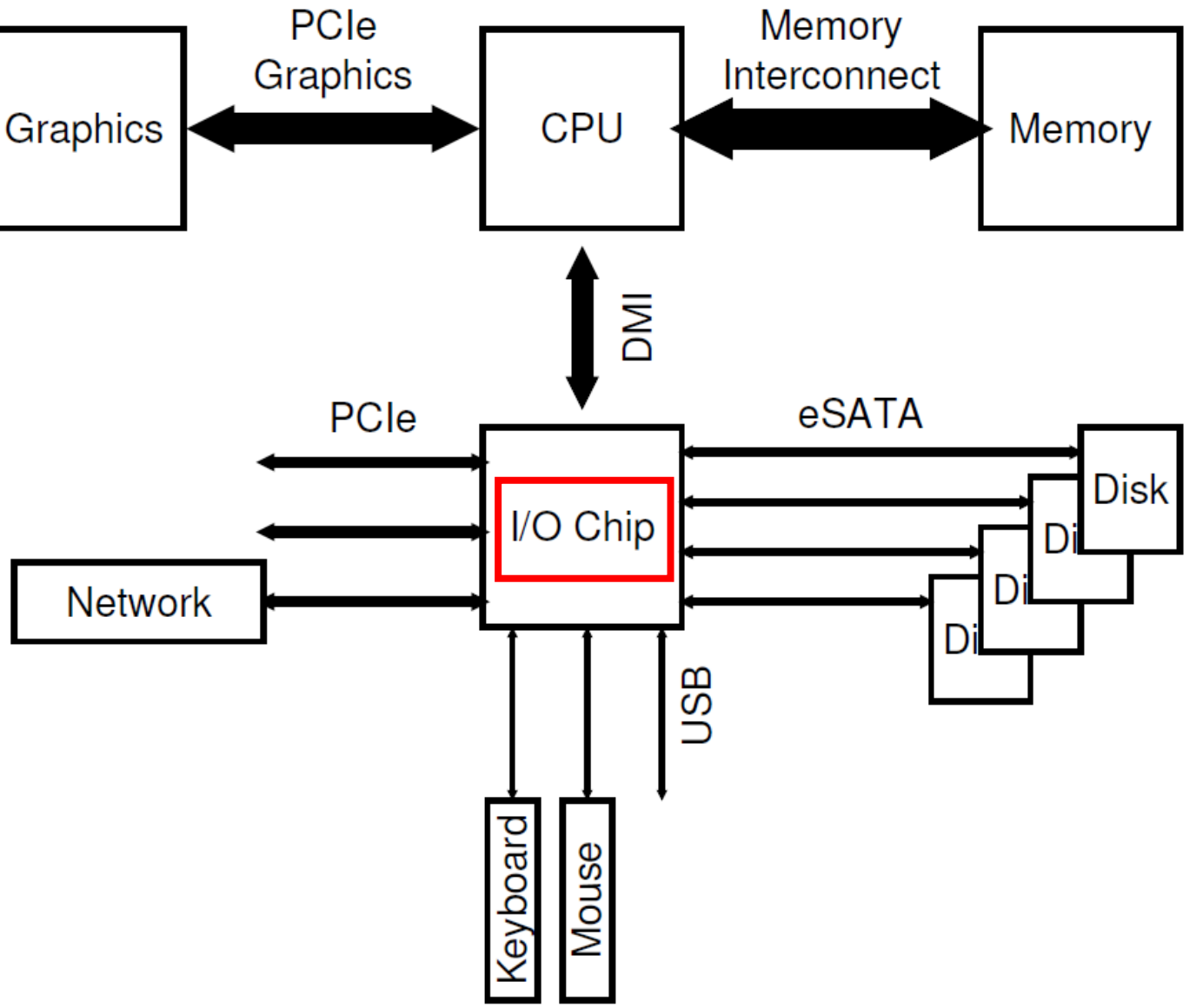
Prototypical System Architecture



Hierarchical Structure: The picture shows a single CPU attached to the main memory of the system via some kind of memory bus or interconnect. Some devices are connected to the system via a general I/O bus, which in many modern systems would be PCI (or one of its many derivatives); graphics and some other higher-performance I/O devices might be found here. Finally, even lower down are one or more of what we call a peripheral bus, such as SCSI, SATA, or USB. These connect slow devices to the system, including disks, mice, and keyboards.

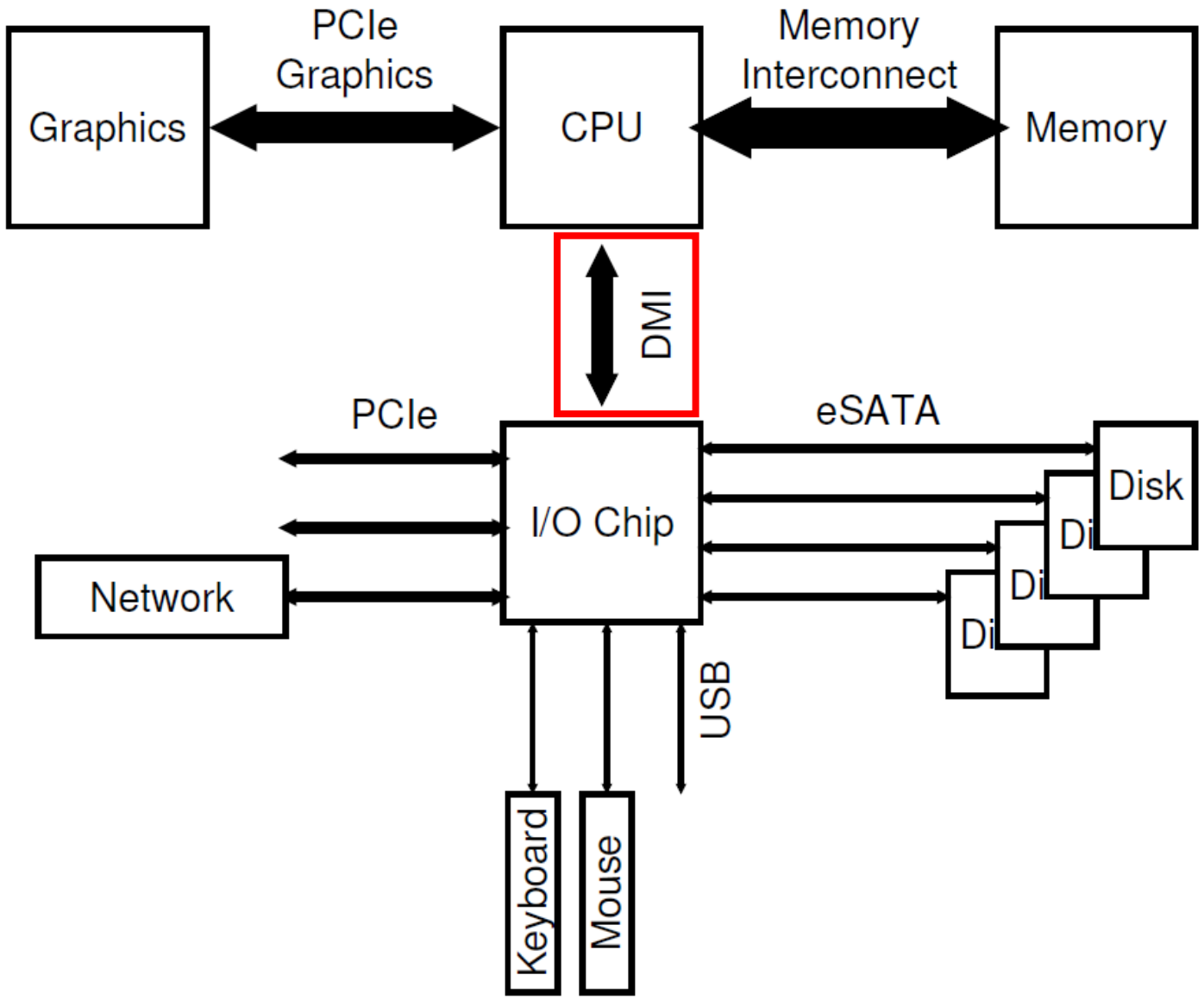
Modern System Architecture

- Modern systems use **specialized chipsets** and faster point-to-point interconnects to improve performance.



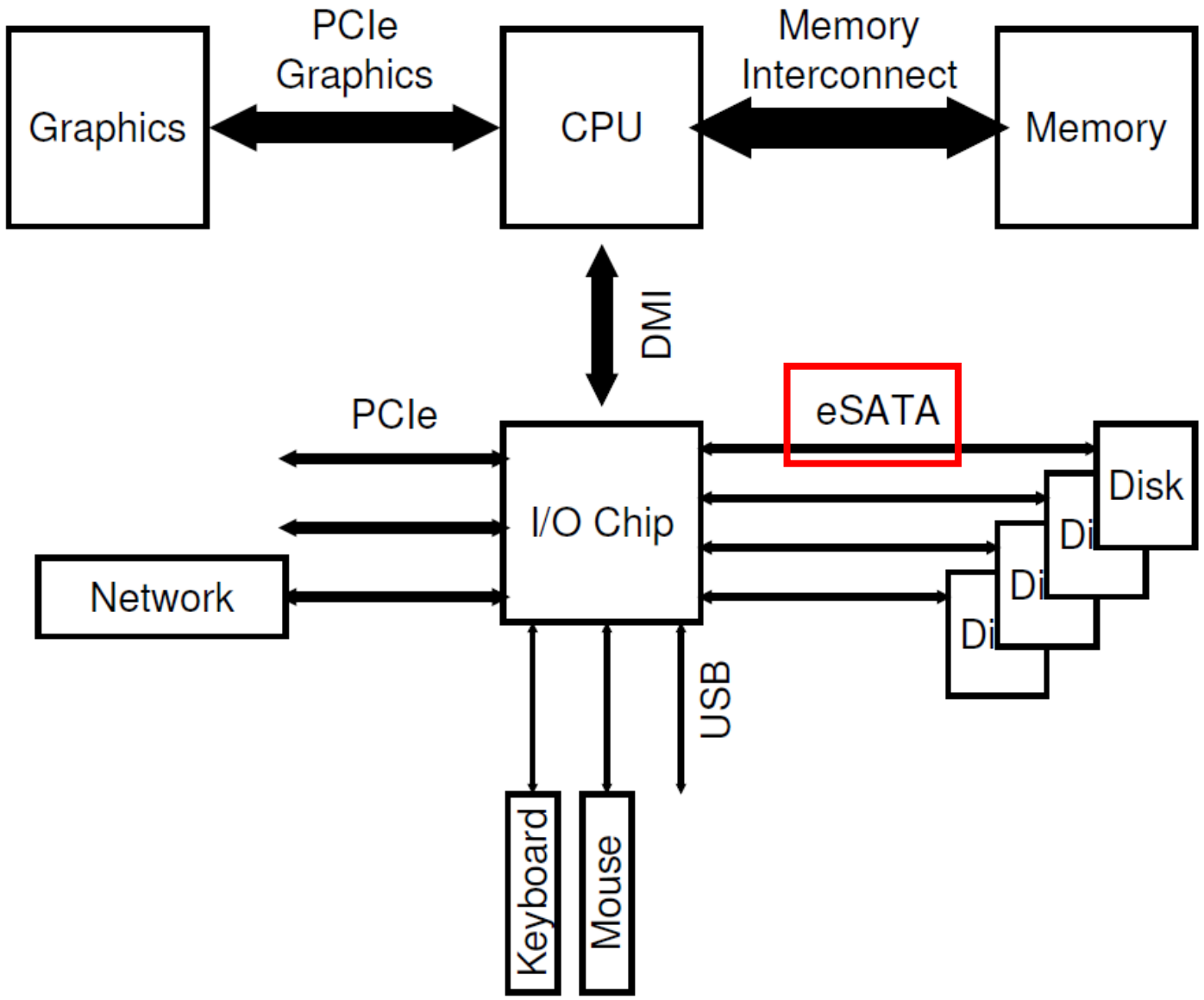
Modern System Architecture

- The CPU connects to an I/O chip via Intel's proprietary **DMI (Direct Media Interface)**, and the rest of the devices connect to this chip via a number of different interconnects.



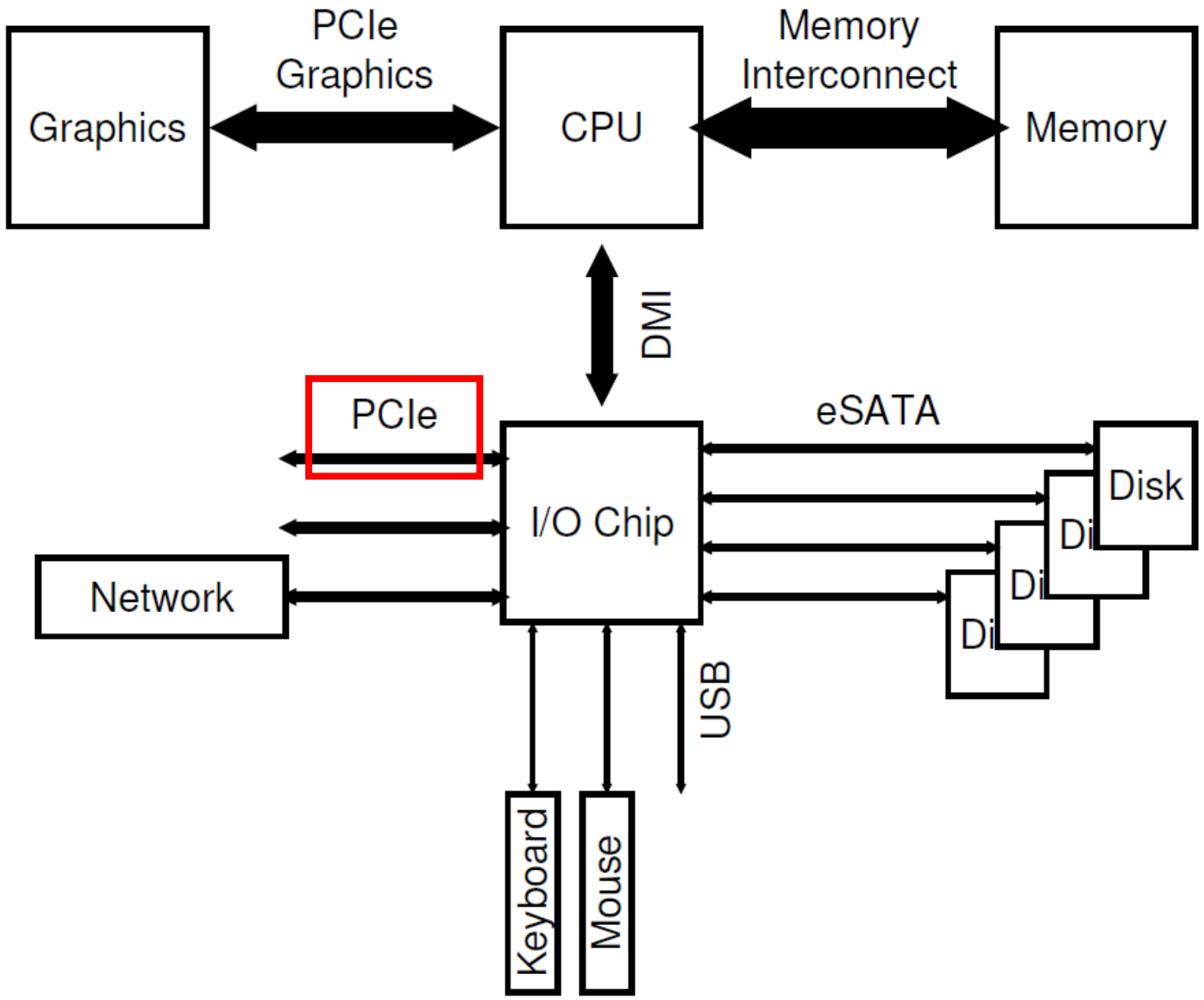
Modern System Architecture

- On the right, one or more hard drives connect to the system via the **eSATA** interface.

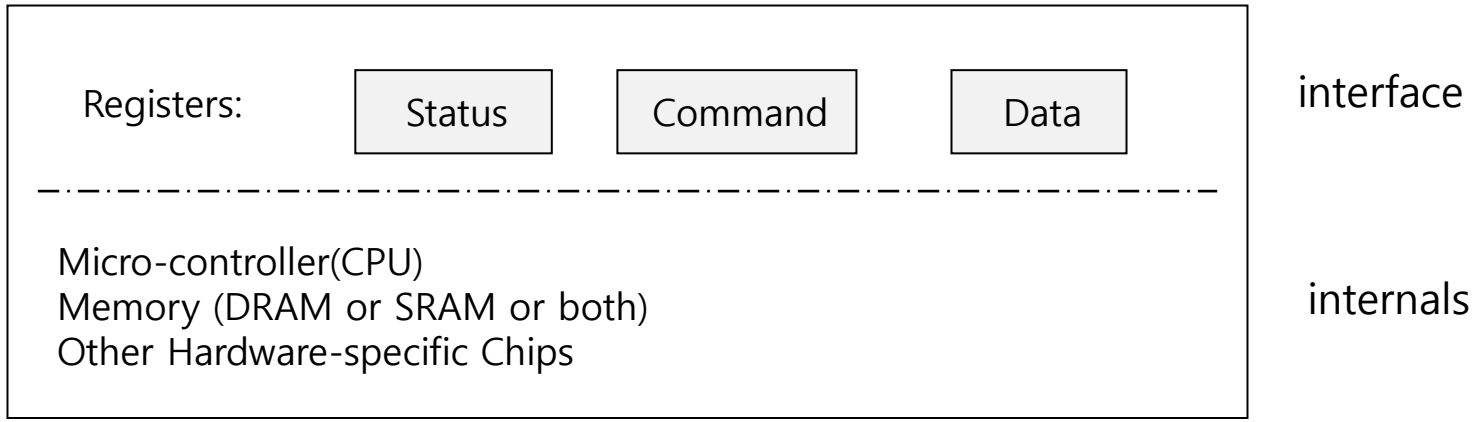


Modern System Architecture

- On the left, other higher performance devices can be connected to the system via **PCIe (Peripheral Component Interconnect Express)**.
- Higher performance storage devices (such as NVMe persistent storage devices) are often connected here.

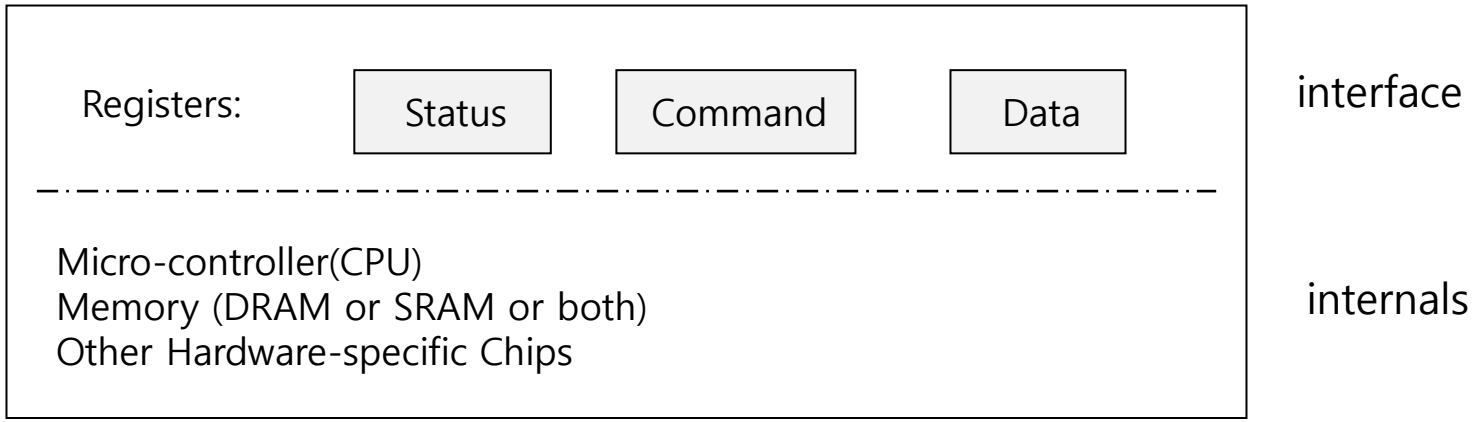


Canonical Device



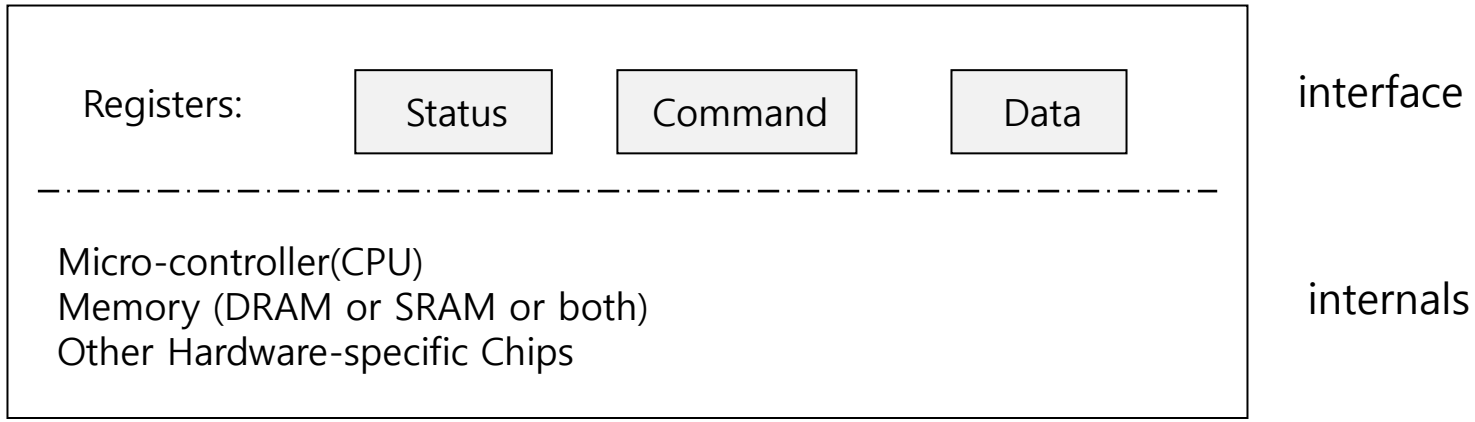
- **Canonical device** (not a real one), helps understand how devices interact.
- The first, is the **hardware interface** it presents that allows the system software to control its operation.
- The second, is its **internal structure**. This part of the device is implementation specific and is responsible for implementing the abstraction the device presents to the system. For instance, the **firmware** (i.e., software within a hardware device) helps implement its functionality.

Canonical Device



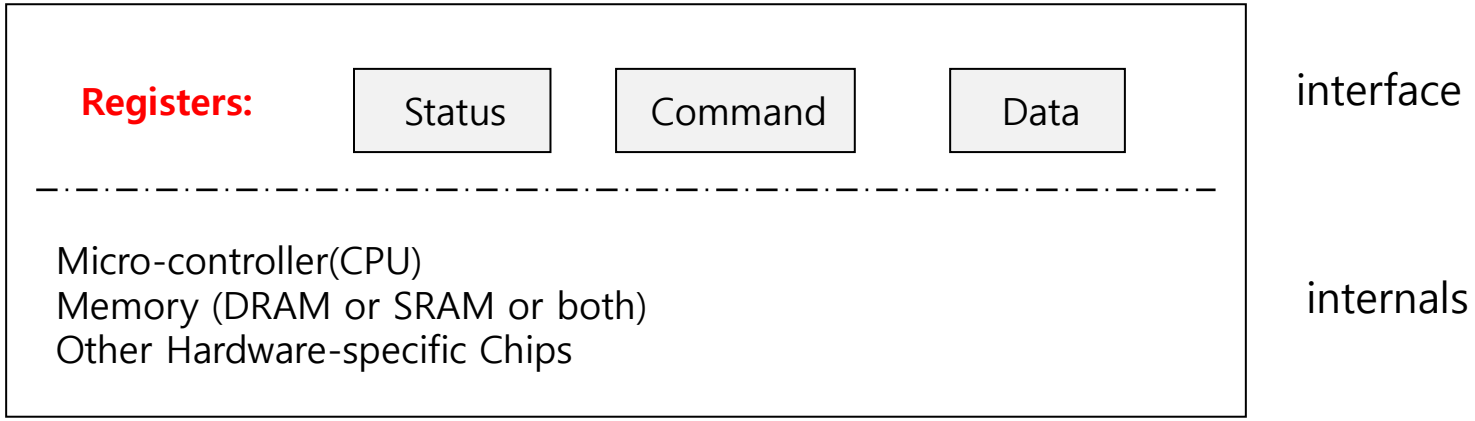
- **Canonical device** (not a real one), helps understand how devices interact.
- The first, is the **hardware interface** it presents that allows the system software to control its operation.
- The second, is its **internal structure**. This part of the device is implementation specific and is responsible for implementing the abstraction the device presents to the system. For instance, the **firmware** (i.e., software within a hardware device) helps implement its functionality.

Canonical Device



- **Canonical device** (not a real one), helps understand how devices interact.
- The first, is the **hardware interface** it presents that allows the system software to control its operation.
- The second, is its **internal structure**. This part of the device is implementation specific and is responsible for implementing the abstraction the device presents to the system. For instance, the **firmware** (i.e., software within a hardware device) helps implement its functionality.

Canonical Device

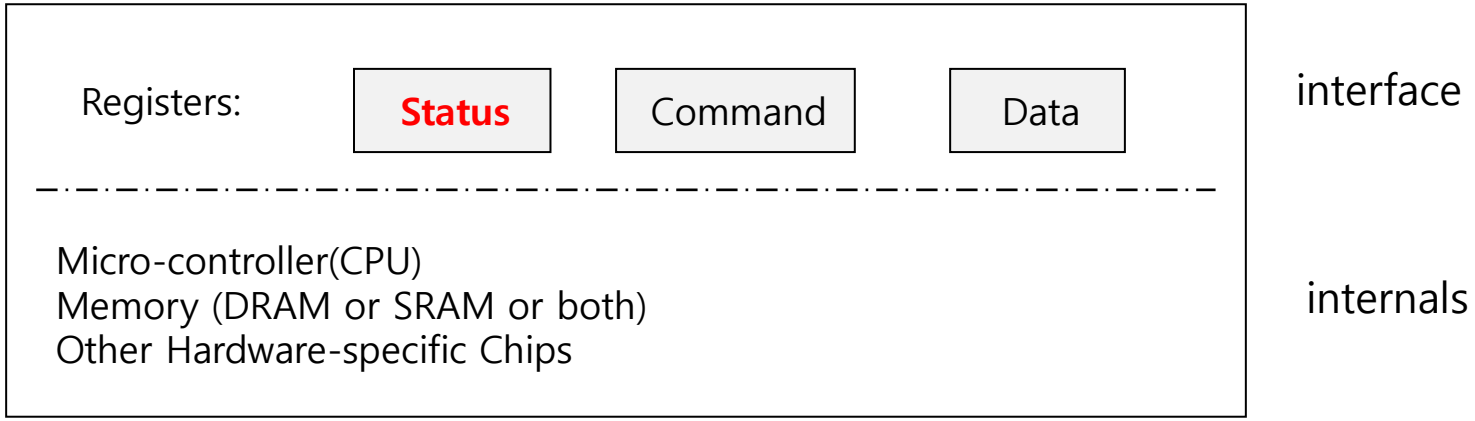


- The (simplified) device interface is comprised of **three registers**:

1. Status register, which can be read to see the current status of the device;
2. Command register, to tell the device to perform a certain task; and
3. Data register to pass data to the device, or get data from the device.

By reading and writing these registers, the operating system can control device behavior.

Canonical Device

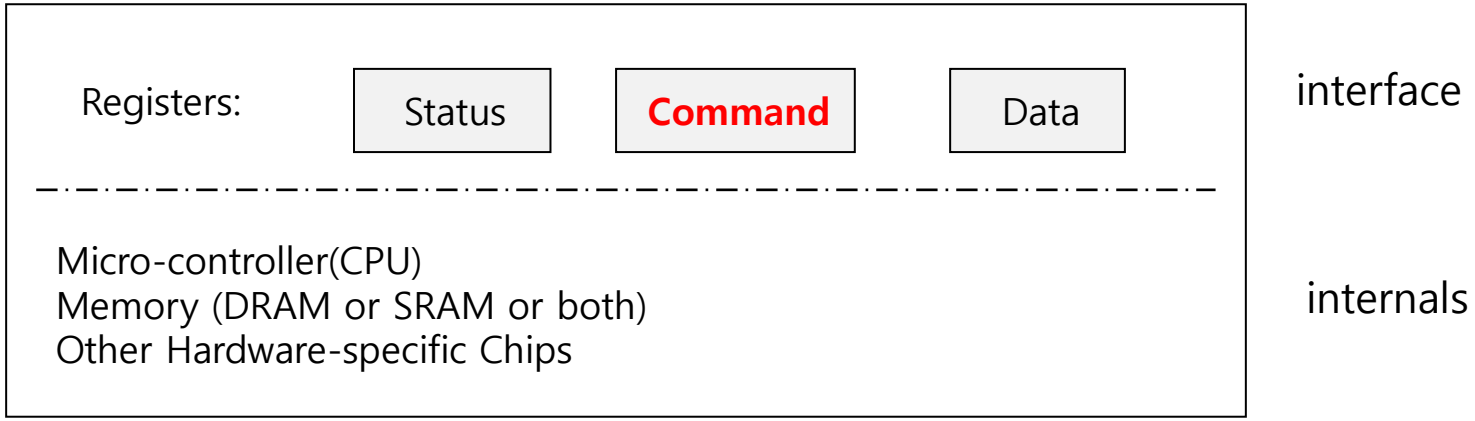


- The (simplified) device interface is comprised of **three registers**:

1. **Status register**, which can be read to see the current status of the device;
2. Command register, to tell the device to perform a certain task; and
3. Data register to pass data to the device, or get data from the device.

By reading and writing these registers, the operating system can control device behavior.

Canonical Device

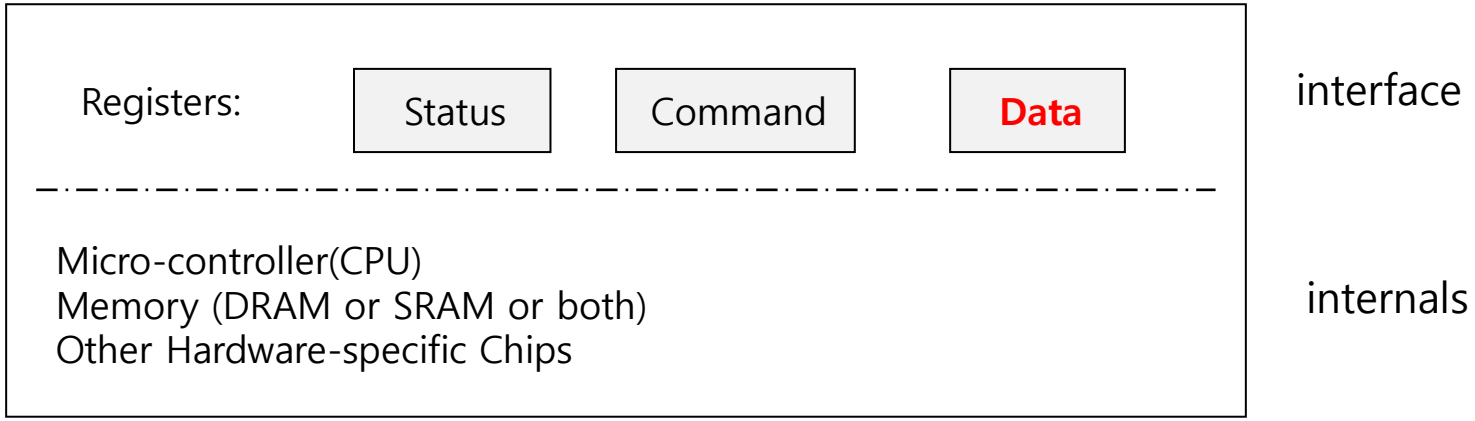


- The (simplified) device interface is comprised of **three registers**:

1. Status register, which can be read to see the current status of the device;
2. **Command register**, to tell the device to perform a certain task; and
3. Data register to pass data to the device, or get data from the device.

By reading and writing these registers, the operating system can control device behavior.

Canonical Device

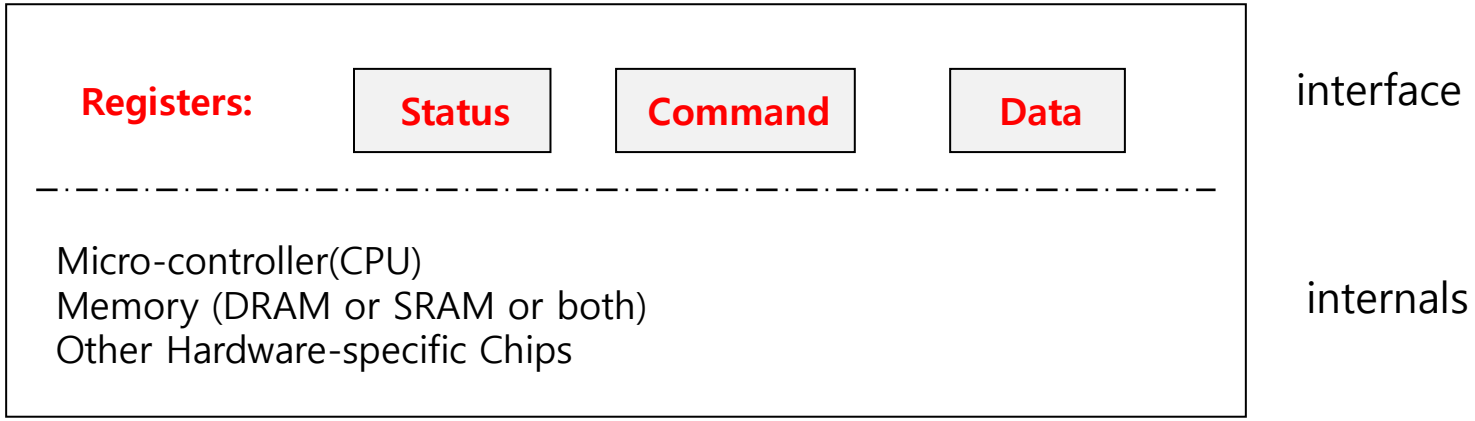


- The (simplified) device interface is comprised of **three registers**:

1. Status register, which can be read to see the current status of the device;
2. Command register, to tell the device to perform a certain task; and
3. **Data register** to pass data to the device, or get data from the device.

By reading and writing these registers, the operating system can control device behavior.

Canonical Device

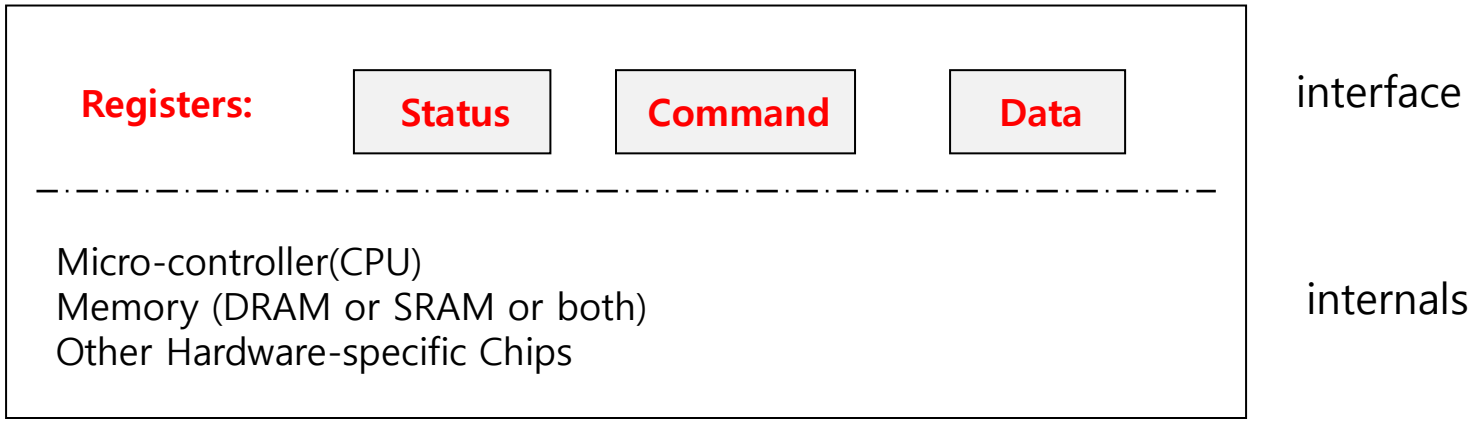


- The (simplified) device interface is comprised of three registers:

1. Status register, which can be read to see the current status of the device;
2. Command register, to tell the device to perform a certain task; and
3. Data register to pass data to the device, or get data from the device.

By reading and writing these registers, the operating system can control device behavior.

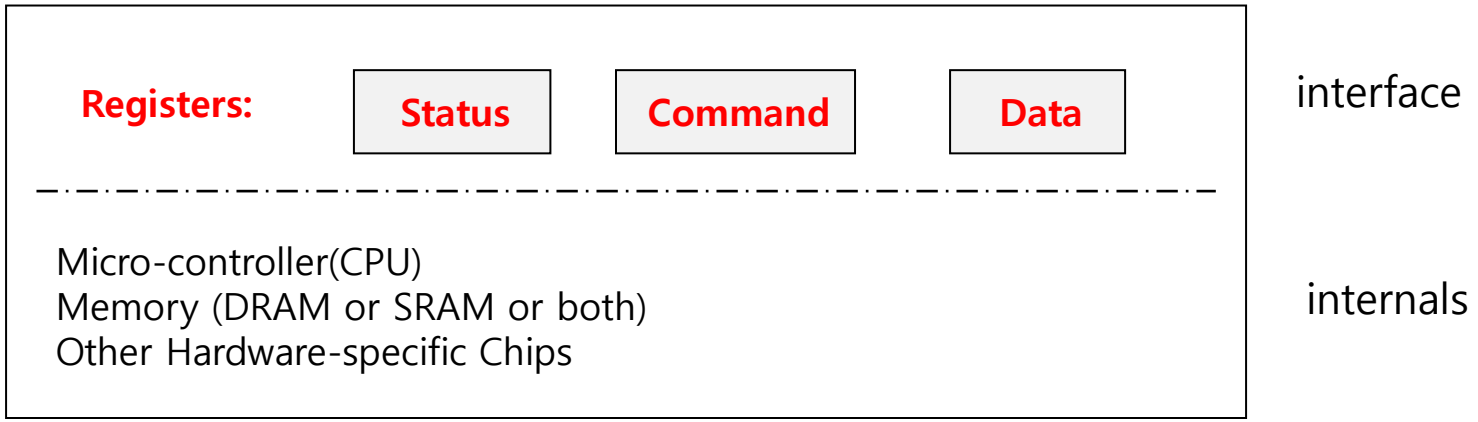
Canonical Device – typical interaction example




```
while ( STATUS == BUSY )
    ; //wait until device is not busy
```

Polling: The OS waits until the device is ready to receive a command, by repeatedly reading the status register.

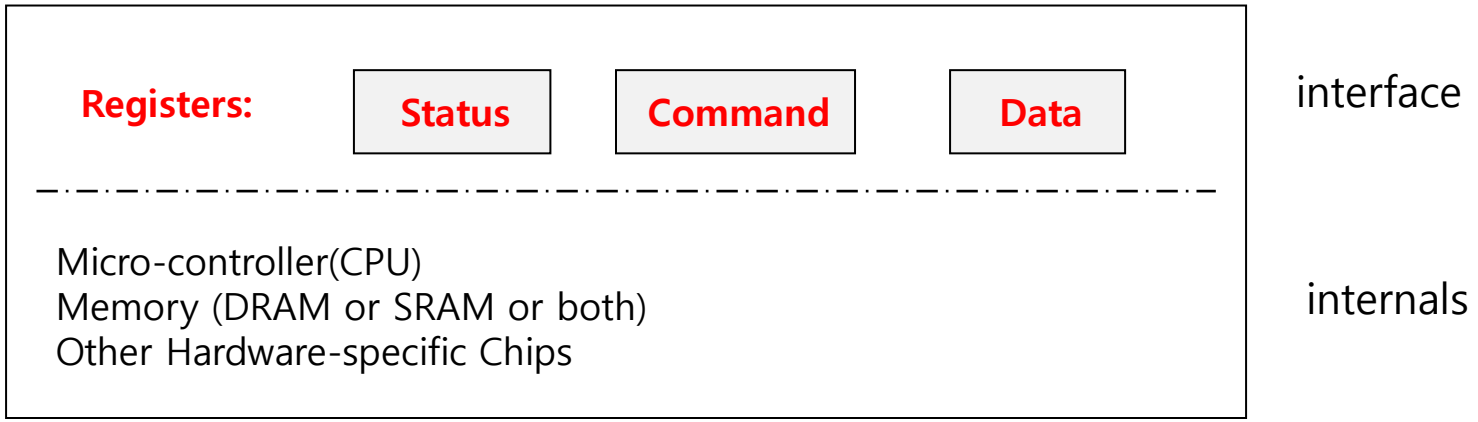
Canonical Device – typical interaction example




```
while ( STATUS == BUSY )
    ; //wait until device is not busy
write data to data register
```

Programmed I/O (PIO): Second, the OS sends some data down to the data register.

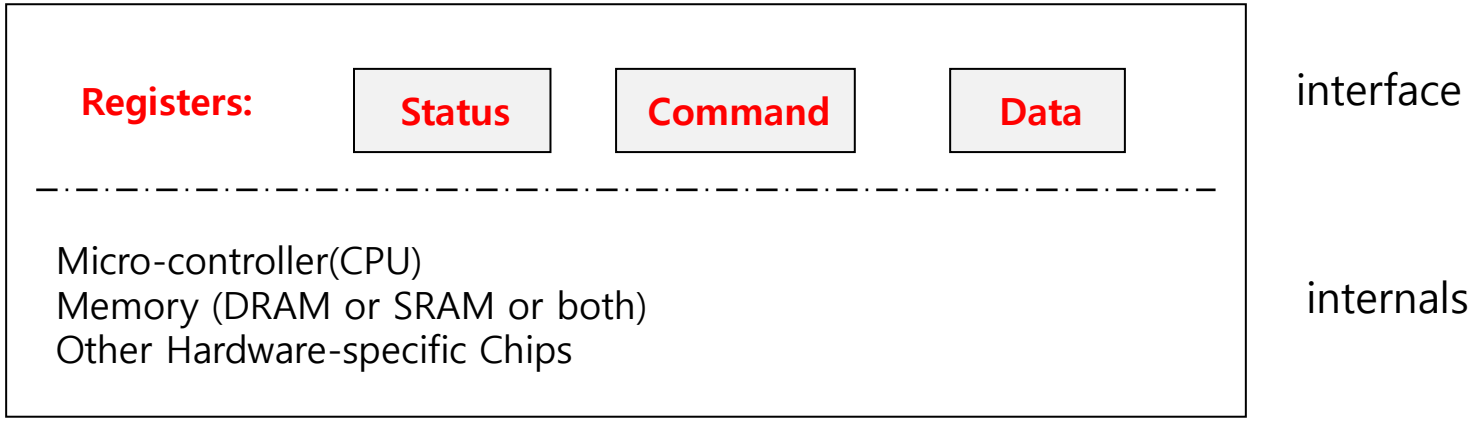
Canonical Device – typical interaction example




```
while ( STATUS == BUSY )
    ; //wait until device is not busy
write data to data register
write command to command register
    Doing so starts the device and executes the command
```

Command: Third, the OS writes a command to the command register; letting the device know that both the data are present and that it should begin working on the command.

Canonical Device – typical interaction example




```
while ( STATUS == BUSY )
    ; //wait until device is not busy
write data to data register
write command to command register
    Doing so starts the device and executes the command
while ( STATUS == BUSY)
    ; //wait until device is done with your request
```

Finally, the OS waits for the device to finish by again polling it in a loop, waiting to see if it is finished (it may then get an error code to indicate success or failure).

Polling

- Operating system waits until the device is ready by **repeatedly** reading the status register.
  - Positive aspect is simple and working.
  - **However, it wastes CPU time just waiting for the device**.
  - Switching to another ready process is better utilizing the CPU.

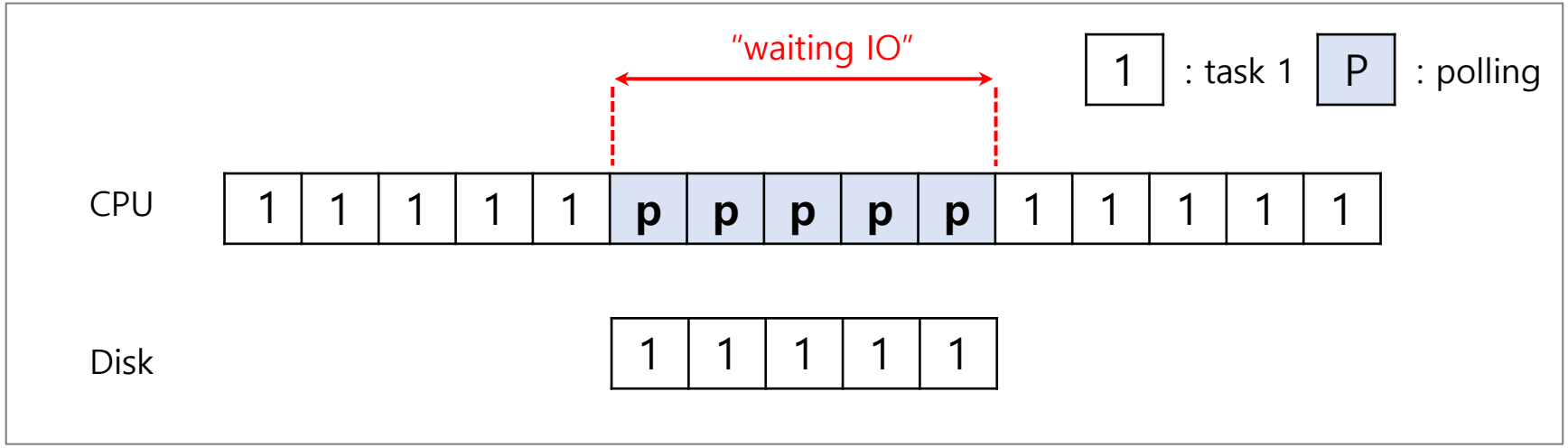


Diagram of CPU utilization by polling

Interrupts

- **Puts the I/O request process to sleep** and switches to another task.
- When the device is finished, wakes the process waiting for the I/O by an **interrupt**.
- More efficient use of **CPU and the disk.**

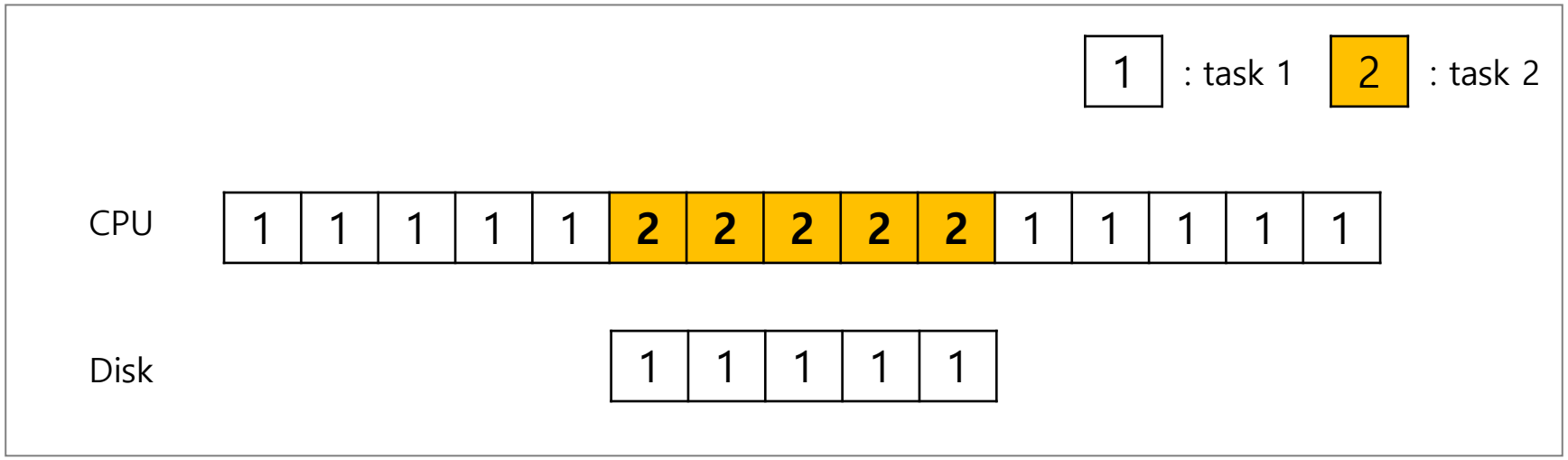


Diagram of CPU utilization by interrupt

Polling vs. Interrupts

- However, **“interrupts is not always the best solution.”**
- It is possible for the OS to “**livelock,”** that is, find itself only processing interrupts and never allowing a user-level process to run and actually service the requests.
- If, device performs very quickly, interrupt will “slow down” the system.
- Because **context switch is expensive (switching to another process)**

If a device is fast → poll is best.
If it is slow → interrupts is better.

Hybrid approach & Coalescing

- **Hybrid**: polls for a little while and then, if the device is not yet finished, uses interrupts.

- Another interrupt-based optimization is **coalescing**. In such a setup, a device which needs to raise an interrupt first waits for a bit before delivering the interrupt to the CPU. While waiting, other requests may soon complete, and thus multiple interrupts can be coalesced into a single interrupt delivery, thus lowering the overhead of interrupt processing.

Over-burdened

CPU wastes a lot of time to copy the a large chunk of data from memory to the device.

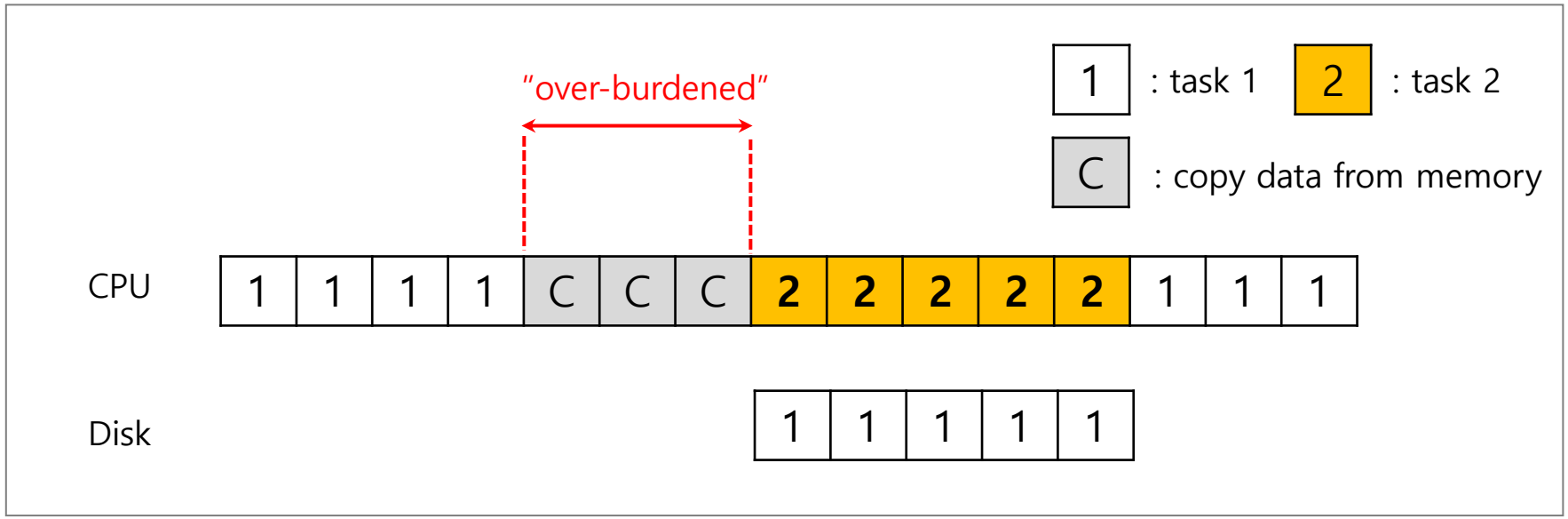


Diagram of CPU utilization

DMA (Direct Memory Access)

- **Copy data** in memory by knowing “where the data lives in memory, how much data to copy.”
- When completed, **DMA raises an interrupt**, I/O begins on Disk.

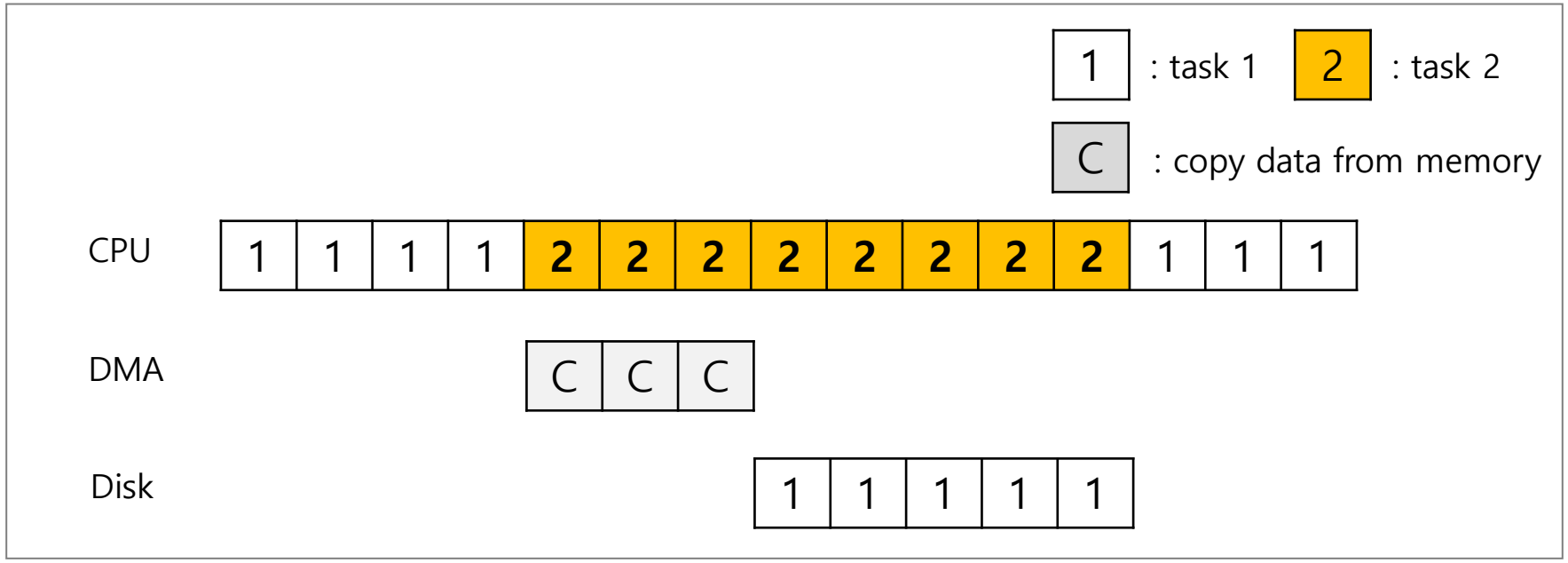


Diagram of CPU utilization by DMA

Device interaction

- How the OS communicates with the device?
- **I/O instructions**: a way for the OS to send data to specific device registers.
  - Example: in and out instructions on x86
  - Such instructions are usually privileged.
- **Memory-mapped I/O**
  - Device registers available as if they were memory locations.
  - The OS load (to read) or store (to write) to the device instead of main memory.

Device interaction

- How the OS communicates with the device?

- **I/O instructions**: a way for the OS to send data to specific device registers.
  - Example: in and out instructions on x86
  - Such instructions are usually privileged.

- **Memory-mapped I/O**
  - Device registers available as if they were memory locations.
  - The OS load (to read) or store (to write) to the device instead of main memory.

Device interaction

- How the OS communicates with the device?

- **I/O instructions**: a way for the OS to send data to specific device registers.
  - Example: in and out instructions on x86
  - Such instructions are usually privileged.

- **Memory-mapped I/O**
  - Device registers available as if they were memory locations.
  - The OS load (to read) or store (to write) to the device instead of main memory.

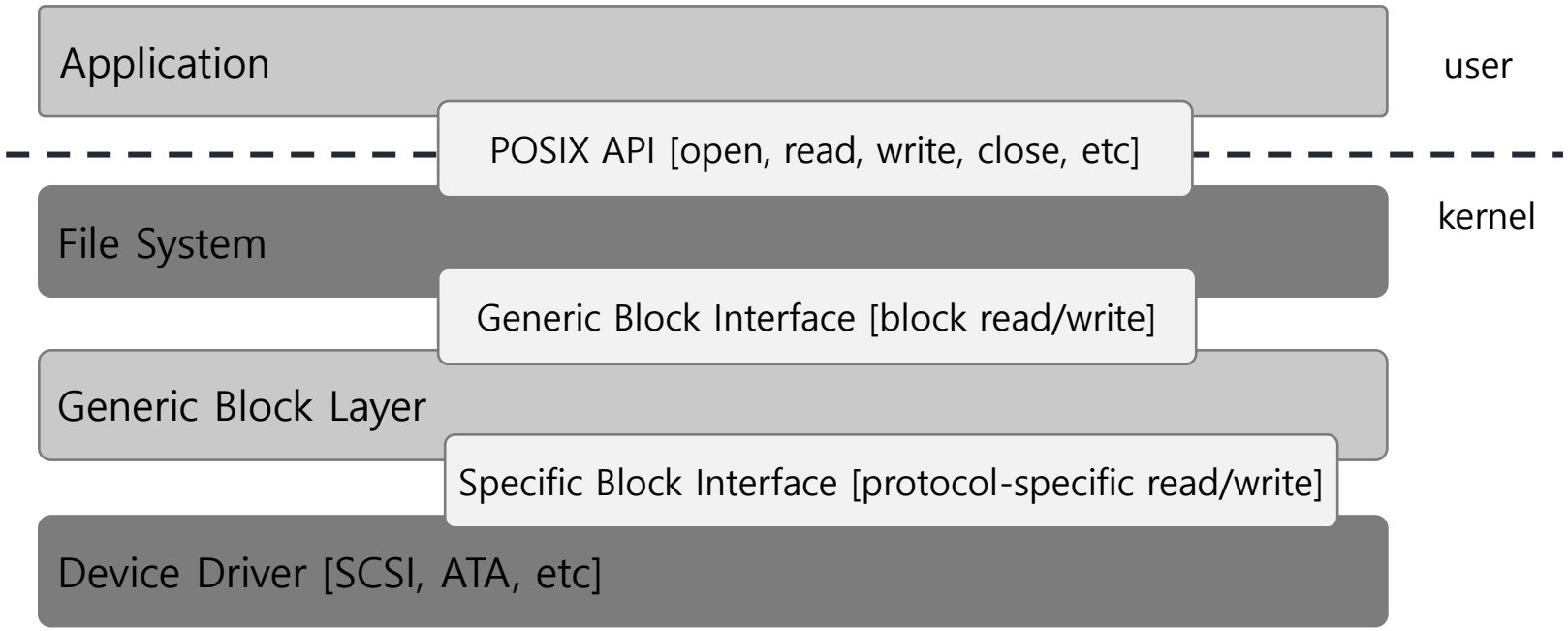
Abstraction

- How the OS interact with different specific interfaces?
  - Ex) We'd like to build a file system that worked on top of SCSI disks, IDE disks, USB keychain drivers, and so on.

- Solutions: Abstraction
  - Abstraction encapsulate any specifics of device interaction.

File System Abstraction

- File system specifics of which disk class it is using.
  - Ex) It issues block read and write request to the generic block layer.



The File System Stack

File System Abstraction – problems

- If there is a device having many special capabilities, these **capabilities will go unused** in the generic interface layer.
  - Example: In Linux with SCSI devices, which have very rich error reporting; because other block devices (e.g., ATA/IDE) have much simpler error handling, all that higher levels of software ever receive is a generic EIO (generic IO error) error code; any extra detail that SCSI may have provided is thus lost to the file system

- Over **70% of OS code is found in device drivers**.
  - Any device drivers are needed because you might plug it to your system.
  - Perhaps more depressingly, as drivers are often written by "amateurs" (instead of full-time kernel developers), they tend to have many more bugs and thus are a primary contributor to kernel crashes.

File System Abstraction – problems

- If there is a device having many special capabilities, these capabilities will go unused in the generic interface layer.
  - Example: In Linux with SCSI devices, which have very rich error reporting; because other block devices (e.g., ATA/IDE) have much simpler error handling, all that higher levels of software ever receive is a generic EIO (generic IO error) error code; any extra detail that SCSI may have provided is thus lost to the file system

- Over **70% of OS code is found in device drivers**.
  - Any device drivers are needed because you might plug it to your system.
  - Perhaps more depressingly, as drivers are often written by "amateurs" (instead of full-time kernel developers), they tend to have many more bugs and thus are a primary contributor to kernel crashes.

A Simple IDE Disk Driver

- Four types of register: Control, command block, status and error
- Memory mapped IO
- in and out I/O instruction

A Simple IDE Disk Driver

- **Control Register**: Address 0x3F6 = 0x80 (0000 1RE0): R=reset, E=0 means "enable interrupt”

- **Command Block Registers**:
  - Address 0x1F0 = Data Port
  - Address 0x1F1 = Error
  - Address 0x1F2 = Sector Count
  - Address 0x1F3 = LBA low byte
  - Address 0x1F4 = LBA mid byte
  - Address 0x1F5 = LBA hi byte
  - Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive
  - Address 0x1F7 = Command/status

A Simple IDE Disk Driver

- **Control Register**: Address 0x3F6 = 0x80 (0000 1RE0): R=reset, E=0 means "enable interrupt”

- **Command Block Registers**:
  - Address 0x1F0 = Data Port
  - Address 0x1F1 = Error
  - Address 0x1F2 = Sector Count
  - Address 0x1F3 = LBA low byte
  - Address 0x1F4 = LBA mid byte
  - Address 0x1F5 = LBA hi byte
  - Address 0x1F6 = 1B1D TOP4LBA: B=LBA, D=drive
  - Address 0x1F7 = Command/status

A Simple IDE Disk Driver

```
Status Register (Address 0x1F7):
    7      6     5      4     3     2     1      0
   BUSY  READY FAULT SEEK  DRQ  CORR IDDEX ERROR

Error Register (Address 0x1F1): (check when ERROR==1)
    7      6     5      4     3     2     1     0
   BBK    UNC   MC    IDNF  MCR  ABRT T0NF AMNF

   BBK  = Bad Block
   UNC  = Uncorrectable data error
   MC   = Media Changed
   IDNF = ID mark Not Found
   MCR  = Media Change Requested
   ABRT = Command aborted
   T0NF = Track 0 Not Found
   AMNF = Address Mark Not Found
```

A Simple IDE Disk Driver

- **Wait for drive to be ready**. Read Status Register until drive is not busy and READY.
- **Write parameters to command registers**. Write the sector count, logical block address (LBA) of the sectors to be accessed, and drive number (master or slave, as IDE permits just two drives) to command registers.
- **Start the I/O**. by issuing read/write to command register. Write READ—WRITE command to command register.
- **Data transfer (for writes)**: Wait until drive status is READY and DRQ (drive request for data); write data to data port.
- **Handle interrupts**. In the simplest case, handle an interrupt for each sector transferred; more complex approaches allow batching and thus one final interrupt when the entire transfer is complete.
- **Error handling**. After each operation, read the status register. If the ERROR bit is on, read the error register for details.

Conclusion

- Dr. B. Wajid
- Wednesday: Aug. 28, 2024